RETAILMART V3 · SQL

SQL Subqueries Part 2 + CTEs

— Siraj Sir

📱 WhatsApp Announcement

SUBQUERIES PART 2 & CTEs

Yesterday: scalar, IN, derived tables. Today: the senior-analyst toolkit. CORRELATED subqueries — inner query references the outer row, runs per-row. EXISTS — fast existence check. CTEs (WITH clause) — named subqueries that read top-to-bottom like a recipe. RECURSIVE CTEs — for hierarchies and self-referencing data. After today, SQL stops being a single command and becomes COMPOSABLE.

Today's Focus:

- Correlated subqueries — inner depends on outer row
- EXISTS / NOT EXISTS — efficient existence check (NULL-safe!)
- CTEs — WITH clause, named subqueries
- Recursive CTEs — hierarchies, trees, date series

earlier closes after this — tomorrow's DB concepts day, then review and capstone prep.
— Siraj Sir
#Day13 #PostgreSQL #CTE #Exists #Correlated

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	Scalar, IN, Derived Tables recap, Where Part 1 was limiting
Part 1: Correlated Subqueries	30 mins	Inner query references outer row, Per-row dynamic comparisons, Performance: runs N times (logically), 4 practice problems (Easy → Crazy)
Part 2: EXISTS / NOT EXISTS	30 mins	Efficient existence check (stops at first match), NOT EXISTS — the safer anti-join (NULL-safe!), EXISTS vs IN — when each wins, 4 practice problems (Easy → Crazy)
Part 3: CTEs (WITH Clause)	35 mins	Named subqueries — read top-to-bottom, Multiple CTEs chained together, Refactoring nested subqueries into CTEs, 4 practice problems (Easy → Crazy)
Part 4: Recursive CTEs	30 mins	Anchor + recursive case + UNION, Date series, number series, tree traversal, Concept walkthrough + 2 practice problems
Quiz + Summary & Wrap-up	10 mins	Quick quiz, Common mistakes, Homework, Database Concepts preview

YOUR SQL JOURNEY

- ✓ Aggregates & GROUP BY
- ✓ JOINs Part 1
- ✓ JOINs Part 2
- ✓ Self Join & UNION
- ✓ Subqueries Part 1
- 13 **Subqueries Part 2 + CTEs** ← HERE
- 14 DB Concepts
- 15 earlier sessions Review

RECAP

Memory Check

Subqueries Part 2 covered three subquery patterns: SCALAR (returns one value), IN (membership in a dynamic list), and DERIVED TABLES (subqueries in FROM). Three workhorse patterns that cover most analyst needs.

But three limitations remain: (1) scalar/derived-table subqueries don't know about the outer row, (2) NOT IN breaks on NULLs, (3) nested subqueries are hard to read at 3+ levels deep. Today fixes all three.

STORY

The Per-Row Benchmark

Your tax form asks: 'Are you in the same income bracket as last year?' To answer this, the tax calculator must look at YOUR salary, then check YOUR previous filings — different numbers for each taxpayer. Every row gets its own comparison. That's exactly a CORRELATED SUBQUERY.

Or imagine a multi-step report — 'first compute monthly revenue, then rank stores within each region, then keep only top 3, then add region totals.' Four logical steps. Trying to nest them all in one query becomes unreadable. CTEs let you write them as named steps that read top-to-bottom like a recipe.

REVEAL

From Story to SQL!

Per-row benchmark → correlated subquery (inner uses outer.column)

'Does at least one match exist?' → EXISTS (stops at first match)

'Find rows with NO matches' → NOT EXISTS (NULL-safe anti-join)

Multi-step logic → WITH cte_name AS (…) chains

Hierarchies / trees → RECURSIVE CTEs (anchor + recursive case)

STORY

The Personalized Average

Yesterday's scalar subquery compared everyone to the COMPANY average. Today's question: 'Find employees above THEIR OWN DEPARTMENT'S average.' That average is DIFFERENT for each employee — it depends on their department. The inner query references the OUTER row. That's CORRELATED.

DEFINITION

Correlated Subquery

Technical:
A correlated subquery references columns from the OUTER query. It runs ONCE PER OUTER ROW (logically — modern optimizers often rewrite this to JOINs). The inner WHERE clause uses outer-row columns to filter.

Layman's:
Correlated = 'for each outer row, ask a CUSTOM question about it.' Each row gets its own dynamic comparison instead of a single global benchmark. Powerful but expensive — runs N times in principle.

SYNTAX: Correlated Subqueries

```
-- Correlated subquery in WHERE
SELECT e.first_name, e.salary, e.dept_id
FROM stores.employees e
WHERE e.salary > (
  SELECT AVG(salary)
  FROM stores.employees inner_e
  WHERE inner_e.dept_id = e.dept_id
    -- ↑ references OUTER table 'e'
);

-- Correlated subquery in SELECT
SELECT c.first_name,
  (SELECT COUNT(*)
   FROM sales.orders o
   WHERE o.cust_id = c.customer_id)
    AS order_count
FROM customers.customers c;

-- The inner WHERE referencing outer = key signal
```

Step-by-Step: Correlated Execution

```
-- Outer query iterates over employees:
-- Row 1: Priya, dept_id=10, salary=70000
--   Inner runs: AVG(salary) WHERE dept_id=10
--   → 65000 → Priya included (70k > 65k)
-- Row 2: Rahul, dept_id=10, salary=60000
--   Inner runs: AVG(salary) WHERE dept_id=10
--   → 65000 → Rahul excluded (60k < 65k)
-- Row 3: Amit, dept_id=20, salary=80000
--   Inner runs: AVG(salary) WHERE dept_id=20
--   → 75000 → Amit included

-- Inner runs per outer row (logically)
-- Modern PostgreSQL often rewrites as a JOIN
```

Easy: Employees Above Their Department's Average

EASY

SCENARIO: The CHRO wants high performers: employees earning above THEIR OWN DEPARTMENT'S average (not the company-wide average). Each employee compared to a DIFFERENT benchmark — the classic correlated subquery use case.

YOUR TASK: Outer query selects employees. Inner correlated subquery: AVG(salary) WHERE dept_id matches outer employee's dept_id. WHERE salary > that dynamic average. Show name, role, dept_id, salary, and the dept_avg.

🤔 **Hint:** Each employee gets a DIFFERENT benchmark (their own department's average) — that's the signature of a correlated subquery. Trap: if the inner query doesn't reference the outer row's dept_id, you'll compute the same company-wide average for everyone, defeating the whole purpose.

Answer

✅ ANSWER

```
SELECT
  e.first_name || ' ' || e.last_name
    AS employee,
  e.role,
  e.dept_id,
  e.salary,
  (SELECT ROUND(AVG(salary), 0)
   FROM stores.employees inner_e
   WHERE inner_e.dept_id = e.dept_id)
    AS dept_avg
FROM stores.employees e
WHERE e.salary > (
  SELECT AVG(salary)
  FROM stores.employees inner_e
  WHERE inner_e.dept_id = e.dept_id
)
ORDER BY e.salary DESC LIMIT 20;
```

Medium: Products Cheaper Than Their Category Average

MEDIUM

SCENARIO: The pricing team wants entry-level products — those priced below their OWN CATEGORY'S average. Each product compared to its category benchmark, not the global average.

YOUR TASK: Products' category lives via dim_brand → dim_category. Correlated subquery: AVG(price) of products in the SAME category as the outer product. WHERE outer.price < that average.

🤔 **Hint:** The inner subquery must JOIN through dim_brand/dim_category to find category siblings — and the correlation point is the OUTER product's category_id. Trap: a correlated subquery doing multiple JOINs PER row can be slow on big tables; if performance matters, a derived table or window function (Window Functions Part 2) is often better.

Answer (1/2)

✅ ANSWER

```
SELECT
  p.product_id,
  p.product_name,
  cat.category_name,
  p.price,
  (SELECT ROUND(AVG(p2.price), 0)
   FROM products.products p2
   JOIN core.dim_brand b2
     ON p2.brand_id = b2.brand_id
   WHERE b2.category_id = b.category_id)
    AS category_avg
FROM products.products p
JOIN core.dim_brand b
  ON p.brand_id = b.brand_id
JOIN core.dim_category cat
  ON b.category_id = cat.category_id
WHERE p.price < (
  SELECT AVG(p2.price)
  FROM products.products p2
  JOIN core.dim_brand b2
    ON p2.brand_id = b2.brand_id
  WHERE b2.category_id = b.category_id
```

Answer (2/2)

✅ ANSWER

```
)
ORDER BY p.price ASC LIMIT 15;
```

Hard: Top Spender Per Region (Correlated MAX)

HARD

SCENARIO: The CMO wants the SINGLE highest-spending customer in each region. For each customer, check if their lifetime spend equals the MAX spend among customers in their region. Correlated subquery returning a per-region MAX.

YOUR TASK: Pre-aggregate customer spend per customer in a derived table (with region). Then for each row, a correlated subquery checks if this customer's spend equals MAX(spend) for their region.

🤔 **Hint:** Two-layer query: pre-aggregate per customer, then filter via correlated subquery. Trap: this exact pattern is the textbook example of a problem that window functions (RANK/ROW_NUMBER on Window Functions Part 1) solve more elegantly — but until then, correlated MAX works fine.

Answer (1/2)

✅ ANSWER

```
SELECT * FROM (
  SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name
      AS customer,
    r.region_name,
    SUM(o.net_total) AS lifetime_spend
  FROM customers.customers c
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
  JOIN stores.stores s
    ON o.store_id = s.store_id
  JOIN core.dim_region r
    ON s.region_id = r.region_id
  GROUP BY c.customer_id,
    c.first_name, c.last_name,
    r.region_name
) AS cust_totals
WHERE lifetime_spend = (
  SELECT MAX(lifetime_spend)
  FROM (
    SELECT
```

Answer (2/2)

✅ ANSWER

```
      c2.customer_id,
      r2.region_name,
      SUM(o2.net_total)
        AS lifetime_spend
    FROM customers.customers c2
    JOIN sales.orders o2
      ON c2.customer_id = o2.cust_id
    JOIN stores.stores s2
      ON o2.store_id = s2.store_id
    JOIN core.dim_region r2
      ON s2.region_id = r2.region_id
    GROUP BY c2.customer_id,
      r2.region_name
  ) AS inner_totals
  WHERE inner_totals.region_name
    = cust_totals.region_name
)
ORDER BY lifetime_spend DESC;
```

Crazy: Brand Diversity Per Customer

CRAZY

SCENARIO: The CMO wants 'brand explorers' — customers who've bought from 5+ distinct brands. For each customer, count their distinct brand purchases. Use a correlated subquery to express the per-customer count, then filter.

YOUR TASK: For each customer, count DISTINCT brand_ids across their orders (correlated subquery walking through orders → order_items → products). Filter WHERE count >= 5. Show name and brand count.

🤔 **Hint:** '5+ distinct brands per customer' = a per-customer count that varies row by row — a correlated subquery is one natural fit. Trap: writing the same correlated COUNT twice (once in SELECT, once in WHERE) makes the database compute it twice — promote it to a CTE that runs once and is referenced twice.

Answer (1/2)

✅ ANSWER

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  (SELECT COUNT(DISTINCT p.brand_id)
   FROM sales.orders o
   JOIN sales.order_items oi
     ON o.order_id = oi.order_id
   JOIN products.products p
     ON oi.prod_id = p.product_id
   WHERE o.cust_id = c.customer_id)
    AS distinct_brands
FROM customers.customers c
WHERE (
  SELECT COUNT(DISTINCT p.brand_id)
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  WHERE o.cust_id = c.customer_id
) >= 5
```

Answer (2/2)

✅ ANSWER

```
ORDER BY distinct_brands DESC LIMIT 15;
```

KEY TAKEAWAYS

Correlated Subqueries

Correlated subquery references the outer query. Runs once per outer row (logically). Used for per-row dynamic comparisons. Inner WHERE referencing an OUTER column is the key signal. Can be slow on large tables — modern optimizers often rewrite to JOINs. CTEs are the readable alternative when the same correlated logic is reused.

COMMON MISTAKE

Correlated vs Independent — Easy to Confuse

The Mistake:
Writing what you think is correlated, but the inner query has no reference to the outer table. Result: same global value substituted for every row — not what you intended.

The Fix:
To be correlated, the inner query MUST reference an outer column. Example: WHERE inner_e.dept_id = e.dept_id where 'e' is the outer alias. Without this reference, you have an independent (constant) subquery.

STORY

The Border Checkpoint

At an international border, the officer doesn't care WHAT documents you have — only whether AT LEAST ONE valid passport exists. That's EXISTS — checks for any matching row, stops at the first match. It doesn't return values, just TRUE or FALSE.

And the visa office's job? Find people who DON'T have a valid passport — NOT EXISTS. The anti-existence check. Critically: handles NULLs cleanly (unlike NOT IN, which silently breaks).

DEFINITION

EXISTS / NOT EXISTS

Technical:
EXISTS (subquery) returns TRUE if the inner query returns AT LEAST ONE row. Stops at the first match — doesn't process more. NOT EXISTS returns TRUE if the inner query returns NO rows. Both are typically used with correlated subqueries.

Layman's:
EXISTS = 'is there at least one matching row?' (returns TRUE/FALSE, doesn't care about values). NOT EXISTS = anti-join, safer than NOT IN. SELECT 1 inside EXISTS is convention — the value is ignored, only existence matters.

SYNTAX: EXISTS / NOT EXISTS

```
-- EXISTS — does at least one row exist?
SELECT c.customer_id, c.first_name
FROM customers.customers c
WHERE EXISTS (
  SELECT 1 FROM sales.orders o
  WHERE o.cust_id = c.customer_id
);  -- ← correlated, references outer c

-- NOT EXISTS — safer anti-join
SELECT c.* FROM customers.customers c
WHERE NOT EXISTS (
  SELECT 1 FROM sales.orders o
  WHERE o.cust_id = c.customer_id
);  -- ← finds customers who never ordered

-- SELECT 1 in EXISTS — convention
-- The value is ignored, only existence matters
```

EXISTS vs IN — Differences

```
-- IN: returns same result, often slower
SELECT * FROM customers c
WHERE c.customer_id IN (
  SELECT cust_id FROM sales.orders
);

-- EXISTS: same result, often FASTER
SELECT * FROM customers c
WHERE EXISTS (
  SELECT 1 FROM sales.orders o
  WHERE o.cust_id = c.customer_id
);

-- Key differences:
-- 1. EXISTS stops at first match — can be faster
-- 2. NOT EXISTS handles NULLs safely (NOT IN doesn't)
-- 3. EXISTS is correlated; IN can use a list
```

Easy: Customers with At Least One Return

EASY

SCENARIO: Customer Service wants the list of customers who have returned at least one product — for satisfaction follow-up. Don't need the return details, just the customer list. EXISTS is the natural fit.

YOUR TASK: Outer: SELECT customers. EXISTS subquery: check if any return exists for this customer (returns → orders → cust_id). Show customer name and email.

🤔 **Hint:** EXISTS just asks 'is there at least one matching row?' — the value selected is irrelevant; SELECT 1 is convention. Trap: EXISTS is correlated by design — the inner query MUST reference the outer row's column; without that reference, the same answer is true for every outer row.

Answer

✅ ANSWER

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  c.email
FROM customers.customers c
WHERE EXISTS (
  SELECT 1
  FROM sales.returns r
  JOIN sales.orders o
    ON r.order_id = o.order_id
  WHERE o.cust_id = c.customer_id
)
ORDER BY c.customer_id
LIMIT 20;
```

Medium: Products Never Bought (NOT EXISTS)

MEDIUM

SCENARIO: The inventory manager needs dead stock: products in the catalog that have NEVER been ordered. NOT EXISTS is the NULL-safe alternative to NOT IN — no risk of zero-row silent failures.

YOUR TASK: Outer: SELECT products. NOT EXISTS subquery: no order_items reference this product. Show product name, brand, price.

🤔 **Hint:** NOT EXISTS is the safer way to express 'finds rows where no matching row exists in another table' — equivalent to NOT IN but immune to the NULL trap. Trap: if you swap NOT EXISTS for NOT IN and any prod_id is NULL, the query silently returns zero rows; NOT EXISTS handles NULL cleanly.

Answer

✅ ANSWER

```
SELECT
  p.product_id,
  p.product_name,
  b.brand_name,
  p.price
FROM products.products p
JOIN core.dim_brand b
  ON p.brand_id = b.brand_id
WHERE NOT EXISTS (
  SELECT 1
  FROM sales.order_items oi
  WHERE oi.prod_id = p.product_id
)
ORDER BY p.price DESC
LIMIT 20;
```

Hard: EXISTS vs IN — Performance and Correctness

HARD

SCENARIO: Your team has two queries that both look for 'customers who placed an order'. Compare them — write both, run both, compare row counts and explanations.

YOUR TASK: Version A: IN-subquery (returns cust_id). Version B: EXISTS with correlated reference to customer_id. Show both — they should return the same rows. Then explain when EXISTS wins.

🤔 **Hint:** Both forms answer 'has this customer ordered?' but EXISTS can stop scanning sales.orders the moment one match is found. Trap: ON LARGE tables with many duplicate cust_ids, EXISTS often wins because IN materializes the full list first; EXPLAIN ANALYZE shows you which strategy the planner picks.

Answer (1/2)

✅ ANSWER

```
-- Version A: IN-subquery
SELECT c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer
FROM customers.customers c
WHERE c.customer_id IN (
  SELECT cust_id
  FROM sales.orders
)
LIMIT 10;

-- Version B: EXISTS (correlated)
SELECT c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer
FROM customers.customers c
WHERE EXISTS (
  SELECT 1
  FROM sales.orders o
  WHERE o.cust_id = c.customer_id
)
LIMIT 10;
```

Answer (2/2)

✅ ANSWER

```
-- Same result. EXISTS often faster
-- on large tables (stops at first match).
```

Crazy: Stores Where NO One Earns Above Average

CRAZY

SCENARIO: HR wants stores where EVERY employee earns BELOW the company average. Use NOT EXISTS: 'there's no employee at this store earning above average'. Nested subquery — NOT EXISTS containing a scalar subquery.

YOUR TASK: Outer: SELECT stores. EXISTS to filter only stores that have employees. NOT EXISTS to require zero above-average employees. The 'above average' uses a scalar subquery for the company-wide mean.

🤔 **Hint:** 'No employee above average exists at this store' is a textbook NOT EXISTS — and the 'above average' part needs its own scalar subquery for the company-wide benchmark. Trap: don't use NOT IN here — if any salary is NULL, NOT IN silently returns zero rows; NOT EXISTS handles NULLs correctly.

Answer (1/2)

✅ ANSWER

```
SELECT
  s.store_id,
  s.store_name,
  s.city,
  (SELECT COUNT(*)
   FROM stores.employees e
   WHERE e.store_id = s.store_id)
    AS employee_count
FROM stores.stores s
WHERE EXISTS (
  SELECT 1
  FROM stores.employees e2
  WHERE e2.store_id = s.store_id
)
AND NOT EXISTS (
  SELECT 1
  FROM stores.employees e
  WHERE e.store_id = s.store_id
    AND e.salary > (
      SELECT AVG(salary)
      FROM stores.employees
    )
```

Answer (2/2)

✅ ANSWER

```
)
ORDER BY employee_count DESC LIMIT 15;
```

KEY TAKEAWAYS

EXISTS / NOT EXISTS

EXISTS checks for AT LEAST ONE match — stops at first match (fast). NOT EXISTS is the safer anti-join — handles NULLs correctly. SELECT 1 inside is convention; the value is ignored. Use NOT EXISTS instead of NOT IN to avoid the NULL trap. EXISTS is always correlated.

COMMON MISTAKE

Forgetting the Correlation

The Mistake:
WHERE EXISTS (SELECT 1 FROM sales.orders) — no reference to outer table. The inner query always returns rows (the table has data), so EXISTS is always TRUE — the filter does nothing.

The Fix:
EXISTS needs a correlation in the inner WHERE: WHERE o.cust_id = c.customer_id. Without it, EXISTS becomes a constant TRUE/FALSE check, not a per-row check.

STORY

The Recipe Steps

A cookbook recipe doesn't say 'in one giant paragraph, mix ingredients, then cook, then plate.' It lists STEPS: '(1) Prep the vegetables. (2) Heat the oil. (3) Sauté. (4) Add sauce.' Each step has a name, builds on the previous one, and the reader follows top-to-bottom.

CTEs do exactly that for SQL. The WITH clause lets you NAME subqueries and chain them. Read top-to-bottom like a recipe instead of inside-out like a nested doll.

DEFINITION

Common Table Expression (CTE)

Technical:

A CTE is a named subquery defined with WITH name AS (...). Available only within the query that defined it. Multiple CTEs can chain — later CTEs can reference earlier ones. Reads top-to-bottom; nested subqueries read inside-out.

Layman's:

CTE = 'name this query result so I can use it later'. Like declaring a variable in code. Same idea as a derived table, but readable and reusable within the same statement. CTEs are the modern way to write complex SQL.

SYNTAX: CTEs (WITH clause)

```
-- Single CTE
WITH top_customers AS (
  SELECT cust_id, SUM(net_total) AS total
  FROM sales.orders
  GROUP BY cust_id
  HAVING SUM(net_total) > 50000
)
SELECT c.first_name, tc.total
FROM top_customers tc
JOIN customers.customers c
  ON tc.cust_id = c.customer_id
ORDER BY tc.total DESC;

-- Multiple CTEs chained
WITH
  step1 AS (SELECT ... FROM ...),
  step2 AS (SELECT ... FROM step1 ...),
  step3 AS (SELECT ... FROM step2 ...)
SELECT * FROM step3;
```

Easy: Top Stores by Revenue (CTE refactor)

EASY

SCENARIO: Yesterday's derived-table query for 'top stores by revenue' worked but was hard to read. Refactor it into a CTE — same result, cleaner code.

YOUR TASK: WITH store_rev AS (SELECT store_id, SUM(net_total)...). Main query: JOIN stores to store_rev. Show top 10 by revenue. The result identical to the derived-table version.

🤔 **Hint:** A CTE is just a derived table with a NAME and pulled out to the top — the rest of the query reads more like steps. Trap: don't forget to alias every aggregate column inside the CTE (SUM(...) AS revenue); without an alias, the outer query can't reference the result.

Answer

✅ ANSWER

```
WITH store_rev AS (
  SELECT
    store_id,
    SUM(net_total) AS revenue,
    COUNT(*) AS order_count
  FROM sales.orders
  GROUP BY store_id
)
SELECT
  s.store_name,
  s.city,
  ROUND(sr.revenue, 0) AS total_revenue,
  sr.order_count
FROM stores.stores s
JOIN store_rev sr
  ON s.store_id = sr.store_id
ORDER BY total_revenue DESC
LIMIT 10;
```

Medium: Churn Analysis — Inactive Customers (6+ Months)

MEDIUM

SCENARIO: The retention team wants 'churned' customers — those whose last order was more than 6 months ago. Use a CTE to compute each customer's last order date, then filter.

YOUR TASK: CTE: customer_last_order — per customer, MAX(order_date). Main query: JOIN to customers, filter last_order < CURRENT_DATE - INTERVAL '6 months'. Show name, last_order, days_since.

🤔 **Hint:** CTEs are perfect for two-step calculations: 'first compute X per customer, then filter on X'. Trap: customers with ZERO orders won't appear in your CTE — if you want them too (they're DEFINITELY churned), use LEFT JOIN in the main query and treat NULL last_order as 'never ordered'.

Answer

✅ ANSWER

```
WITH customer_last_order AS (
  SELECT
    cust_id,
    MAX(order_date) AS last_order
  FROM sales.orders
  GROUP BY cust_id
)
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  c.email,
  clo.last_order,
  (CURRENT_DATE - clo.last_order)
    AS days_since
FROM customers.customers c
JOIN customer_last_order clo
  ON c.customer_id = clo.cust_id
WHERE clo.last_order
  < CURRENT_DATE - INTERVAL '6 months'
ORDER BY clo.last_order ASC
LIMIT 20;
```

Hard: Refactor a 3-Level Nested Subquery into Chained CTEs

HARD

SCENARIO: A junior wrote a 3-level nested subquery for 'top 5 categories by revenue, with their store count and customer count'. It works but is unreadable. Refactor it into 3 chained CTEs that read top-to-bottom.

YOUR TASK: CTE 1: category_revenue (category → total revenue). CTE 2: category_stores (category → unique store count). CTE 3: category_customers (category → unique customer count). Main query: JOIN all three by category, ORDER BY revenue, LIMIT 5.

🤔 **Hint:** Three independent aggregations → three CTEs joined at the end. Trap: each CTE must group by the SAME key (category) so the final JOIN aligns; if one CTE groups by category_id and another by category_name, the JOIN gets messy — pick one key and stick with it.

Answer (1/3)

✅ ANSWER

```
WITH
  category_revenue AS (
    SELECT b.category_id,
      cat.category_name,
      SUM(oi.net_amount) AS revenue
    FROM sales.order_items oi
    JOIN products.products p
      ON oi.prod_id = p.product_id
    JOIN core.dim_brand b
      ON p.brand_id = b.brand_id
    JOIN core.dim_category cat
      ON b.category_id = cat.category_id
    GROUP BY b.category_id,
      cat.category_name),
  category_stores AS (
    SELECT b.category_id,
      COUNT(DISTINCT o.store_id)
        AS store_count
    FROM sales.orders o
    JOIN sales.order_items oi
      ON o.order_id = oi.order_id
    JOIN products.products p
```

Answer (2/3)

✅ ANSWER

```
      ON oi.prod_id = p.product_id
    JOIN core.dim_brand b
      ON p.brand_id = b.brand_id
    GROUP BY b.category_id),
  category_customers AS (
    SELECT b.category_id,
      COUNT(DISTINCT o.cust_id)
        AS customer_count
    FROM sales.orders o
    JOIN sales.order_items oi
      ON o.order_id = oi.order_id
    JOIN products.products p
      ON oi.prod_id = p.product_id
    JOIN core.dim_brand b
      ON p.brand_id = b.brand_id
    GROUP BY b.category_id)
SELECT
  cr.category_name,
  ROUND(cr.revenue, 0) AS revenue,
  cs.store_count,
  cc.customer_count
FROM category_revenue cr
```

Answer (3/3)

✅ ANSWER

```
JOIN category_stores cs
  ON cr.category_id = cs.category_id
JOIN category_customers cc
  ON cr.category_id = cc.category_id
ORDER BY cr.revenue DESC LIMIT 5;
```

Crazy: VIP Engagement Score — Multi-Step CTE Chain

CRAZY

SCENARIO: The CMO wants a VIP engagement score per customer: (1) total spend, (2) recency in days, (3) brand diversity, (4) review activity. Combine all four into ONE score, then return the top 10. Four CTEs feeding a fifth that computes the composite score.

YOUR TASK: Five CTEs: spend, recency, brands, reviews, scored. The 'scored' CTE LEFT JOINs the first four by customer_id, computes a weighted score. Main query: order by score, return top 10 with all components.

🤔 **Hint:** When a query has 4+ logical stages, CTEs become essential for readability — nested subqueries would be unreadable. Trap: use LEFT JOIN when chaining CTEs together if some customers might be missing from intermediate CTEs (e.g., customers with no reviews still exist — don't lose them with INNER JOIN).

Answer (1/3)

✅ ANSWER

```
WITH
  spend AS (
    SELECT cust_id,
      SUM(net_total) AS total_spend
    FROM sales.orders
    GROUP BY cust_id),
  recency AS (
    SELECT cust_id,
      CURRENT_DATE - MAX(order_date)
        AS days_since
    FROM sales.orders
    GROUP BY cust_id),
  brands AS (
    SELECT o.cust_id,
      COUNT(DISTINCT p.brand_id)
        AS brand_count
    FROM sales.orders o
    JOIN sales.order_items oi
      ON o.order_id = oi.order_id
    JOIN products.products p
      ON oi.prod_id = p.product_id
    GROUP BY o.cust_id),
```

Answer (2/3)

✅ ANSWER

```
reviews AS (
  SELECT customer_id AS cust_id,
    COUNT(*) AS review_count
  FROM customers.reviews
  GROUP BY customer_id),
scored AS (
  SELECT s.cust_id,
    s.total_spend,
    r.days_since,
    b.brand_count,
    COALESCE(rv.review_count, 0)
      AS review_count,
    (s.total_spend / 1000.0
     + b.brand_count * 5
     + COALESCE(rv.review_count, 0) * 10
     - r.days_since * 0.1)
      AS score
  FROM spend s
  JOIN recency r ON s.cust_id = r.cust_id
  JOIN brands b ON s.cust_id = b.cust_id
  LEFT JOIN reviews rv
    ON s.cust_id = rv.cust_id)
```

Answer (3/3)

✅ ANSWER

```
SELECT
  c.first_name || ' ' || c.last_name
    AS customer,
  ROUND(sc.total_spend, 0) AS spend,
  sc.days_since,
  sc.brand_count,
  sc.review_count,
  ROUND(sc.score, 1) AS score
FROM scored sc
JOIN customers.customers c
  ON sc.cust_id = c.customer_id
ORDER BY sc.score DESC LIMIT 10;
```

KEY TAKEAWAYS

CTEs

CTE = named subquery with WITH name AS (...). Reads top-to-bottom like a recipe. Multiple CTEs chain — later ones reference earlier ones. Modern replacement for nested subqueries. Same idea as derived tables but more readable and reusable. PostgreSQL inlines CTEs by default (12+), so performance is competitive with derived tables.

REAL WORLD

CTEs in Production

Every modern analyst codebase is full of CTEs:

- dbt (data build tool) ENTIRELY built on CTEs — each model is a CTE chain
- Looker / Mode / Hex — interactive analytics tools generate CTE-heavy SQL
- Capstone projects in this course will use CTEs in 90%+ of queries
- Modern interviewers EXPECT candidates to write CTEs over nested subqueries

Once you start writing CTEs, you'll wonder how you ever wrote nested SQL.

PRO TIP

PRO TIP

CTE vs derived table: identical from the DB's perspective (PostgreSQL 12+ inlines CTEs by default). Choose CTE when: (1) the logic is multi-step, (2) you reference the same subquery twice, or (3) readability matters. Choose derived table when: the subquery is small and used once inline.

STORY

Walking Up a Family Tree

'List all ancestors of person X.' To answer, you find X's parents, then THEIR parents, then THEIRS — walking up the tree until you reach the root. The logic is REPETITIVE: at each step, apply the same rule to the previous result.

That's a RECURSIVE CTE. Two parts: an ANCHOR (starting point) and a RECURSIVE step (apply this transformation, then keep going). UNION ALL combines all generations into one result. Used for: org charts, category trees, bill-of-materials, date series, number sequences.

DEFINITION

Recursive CTE

Technical:
WITH RECURSIVE cte AS (anchor_query UNION ALL recursive_query). The anchor returns the starting rows. The recursive query references the CTE itself and produces the next 'generation'. PostgreSQL repeats until the recursive query returns no rows. UNION ALL stacks all generations.

Layman's:
Recursive CTE = 'start with X, then repeatedly apply this rule until nothing new is produced.' Like a snowball rolling down a hill — each step builds on the previous. Used for any 'walk through a tree' or 'generate a series' problem.

SYNTAX: Recursive CTE

```
-- Anatomy:
WITH RECURSIVE cte_name AS (
  -- Anchor: starting rows
  SELECT col1, col2, 1 AS depth
  FROM source
  WHERE starting_condition

  UNION ALL

  -- Recursive: builds on previous iteration
  SELECT child.col1, child.col2, parent.depth + 1
  FROM source child
  JOIN cte_name parent
    ON child.parent_id = parent.id
  -- May include termination condition
)
SELECT * FROM cte_name;
```

Step-by-Step: Recursive Execution

```
-- Example: count 1 to 5
WITH RECURSIVE counter AS (
  SELECT 1 AS n            -- Anchor: row (1)
  UNION ALL
  SELECT n + 1              -- Recursive step
  FROM counter
  WHERE n < 5               -- Stop condition
)
SELECT * FROM counter;

-- Step 1: Anchor → {1}
-- Step 2: Recursive on {1} → {2}
-- Step 3: Recursive on {2} → {3}
-- Step 4: Recursive on {3} → {4}
-- Step 5: Recursive on {4} → {5}
-- Step 6: Recursive on {5} → {} (WHERE 5<5 FALSE)
-- Final: UNION ALL of {1},{2},{3},{4},{5} = 1..5
```

Easy: Generate a Number Series 1 to 100

EASY

SCENARIO: You need a list of integers 1-100 (without depending on any table). Recursive CTEs can build series — the simplest demonstration of how anchor + recursive step works.

YOUR TASK: Anchor: SELECT 1. Recursive: SELECT n+1 FROM counter WHERE n < 100. UNION ALL combines them. Return all 100 rows.

🤔 **Hint:** Recursive CTE = anchor (the seed) + UNION ALL + recursive step (referencing the CTE itself). Trap: forgetting the WHERE termination condition causes infinite recursion — PostgreSQL eventually errors out, but the query may hang for a while first.

Answer

✅ ANSWER

```
WITH RECURSIVE counter AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1
  FROM counter
  WHERE n < 100
)
SELECT n FROM counter
ORDER BY n;
```

Medium: Date Series for a Calendar Table

MEDIUM

SCENARIO: Analytics needs a daily date list for the last 30 days — useful for 'fill in zeros for missing days' reports. Recursive CTE builds a date sequence by stepping +1 day each iteration.

YOUR TASK: Anchor: CURRENT_DATE - 29 (start 30 days ago). Recursive: previous_date + 1. Stop at CURRENT_DATE. Return all 30 dates. Add day-of-week column for bonus.

🤔 **Hint:** Date arithmetic in PostgreSQL: date + integer = date with N days added. Trap: PostgreSQL also has generate_series for this exact purpose — usually preferred; the recursive CTE here is for LEARNING the pattern, not because it's the best tool for date series in production.

Answer

✅ ANSWER

```
WITH RECURSIVE date_series AS (
  SELECT CURRENT_DATE - 29 AS day,
    TO_CHAR(CURRENT_DATE - 29, 'Day')
      AS day_name
  UNION ALL
  SELECT day + 1,
    TO_CHAR(day + 1, 'Day')
  FROM date_series
  WHERE day < CURRENT_DATE
)
SELECT day, TRIM(day_name) AS day_name
FROM date_series
ORDER BY day;
```

Hard: Cumulative Order Days Per Customer

HARD

SCENARIO: For each customer, generate every day from their first order to the current date — useful for cohort retention analysis (need rows for empty days too).

YOUR TASK: Anchor CTE: customer + first_order_date. Recursive CTE: walk one day forward until current date. JOIN back to customers for the name. Return customer, day_number (1, 2, 3…), date.

🤔 **Hint:** Recursive CTEs can use values from the OUTER world as anchors — here, each customer's first order date. Trap: this query can produce HUGE outputs (millions of rows for many customers × many days) — always LIMIT in practice or filter to a small set of customers for demo.

Answer (1/2)

✅ ANSWER

```
WITH RECURSIVE customer_days AS (
  SELECT
    cust_id,
    MIN(order_date) AS day,
    1 AS day_num
  FROM sales.orders
  WHERE cust_id <= 5
  GROUP BY cust_id
  UNION ALL
  SELECT
    cust_id,
    day + 1,
    day_num + 1
  FROM customer_days
  WHERE day < CURRENT_DATE
    AND day_num < 60  -- safety limit
)
SELECT
  c.first_name AS customer,
  cd.day,
  cd.day_num
FROM customer_days cd
```

Answer (2/2)

✅ ANSWER

```
JOIN customers.customers c
  ON cd.cust_id = c.customer_id
ORDER BY cd.cust_id, cd.day
LIMIT 30;
```

Crazy: Walk a Synthetic Org Chart

CRAZY

SCENARIO: RetailMart's stores.employees table has no manager_id column — so we can't walk a real hierarchy. Instead: BUILD a synthetic 3-level org tree from VALUES, then use a recursive CTE to traverse from the CEO downward. This is the canonical 'org chart' interview question, adapted to data we have.

YOUR TASK: Create a synthetic VALUES table with employee_id, name, manager_id. Anchor: rows where manager_id IS NULL (the CEO). Recursive: JOIN children to parents. Show name, level (depth from root), and path.

🤔 **Hint:** Recursive org-chart pattern: anchor = root rows (NULL parent), recursive step = JOIN children to the running CTE. Trap: building 'path' as a concatenated string ('CEO > VP > Mgr > Staff') requires keeping the path column in BOTH the anchor and recursive step — and casting carefully to match types.

Answer (1/2)

✅ ANSWER

```
WITH RECURSIVE
  org (id, name, mgr_id, lvl, path) AS (
    -- Synthetic org chart
    SELECT id, name, mgr_id, 1 AS lvl,
      name::TEXT AS path
    FROM (VALUES
      (1, 'Asha (CEO)', NULL),
      (2, 'Bina (VP Sales)', 1),
      (3, 'Chetan (VP Ops)', 1),
      (4, 'Diya (Sales Mgr)', 2),
      (5, 'Esha (Ops Mgr)', 3),
      (6, 'Farhan (Rep)', 4),
      (7, 'Gita (Rep)', 4),
      (8, 'Hari (Staff)', 5)
    ) AS emp(id, name, mgr_id)
    WHERE mgr_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.mgr_id,
      o.lvl + 1,
      o.path || ' > ' || e.name
    FROM (VALUES
      (1, 'Asha (CEO)', NULL),
```

Answer (2/2)

✅ ANSWER

```
      (2, 'Bina (VP Sales)', 1),
      (3, 'Chetan (VP Ops)', 1),
      (4, 'Diya (Sales Mgr)', 2),
      (5, 'Esha (Ops Mgr)', 3),
      (6, 'Farhan (Rep)', 4),
      (7, 'Gita (Rep)', 4),
      (8, 'Hari (Staff)', 5)
    ) AS e(id, name, mgr_id)
    JOIN org o ON e.mgr_id = o.id
  )
SELECT lvl, name, path
FROM org
ORDER BY lvl, name;
```

KEY TAKEAWAYS

Recursive CTEs

Recursive CTE = anchor (seed rows) + UNION ALL + recursive step (references CTE itself). PostgreSQL repeats until the recursive step returns zero rows. Used for: hierarchies (org charts, category trees), series (dates, numbers), tree traversal. ALWAYS include a termination condition. Add a safety counter (lvl < 100) to guard against runaway recursion.

COMMON MISTAKE

Infinite Recursion

The Mistake:
Recursive step with no termination condition. PostgreSQL keeps recursing — eventually errors out, but the query runs for a while first. Or worse: a cycle in the data (employee A reports to B reports to A) causes infinite recursion.

The Fix:
Always include a stop condition: WHERE day < CURRENT_DATE, or AND depth < 100. For cyclic data, use UNION (which deduplicates) instead of UNION ALL — costs more but breaks cycles.

STORY

Quick Fire Round!

Modify each query. Convert EXISTS to NOT EXISTS. Add another CTE step. Increase the recursion depth. Understanding comes from experimentation.

Challenge 1: EXISTS Speed Demo

```
-- Customers who have ordered (EXISTS)
SELECT c.first_name FROM customers.customers c
WHERE EXISTS (
  SELECT 1 FROM sales.orders o
  WHERE o.cust_id = c.customer_id
) LIMIT 10;

-- YOUR TURN: Rewrite as IN-subquery,
-- compare EXPLAIN ANALYZE timings
```

Challenge 2: Correlated Per-Customer Count

```
-- Each customer with their order count
SELECT c.first_name,
  (SELECT COUNT(*)
   FROM sales.orders o
   WHERE o.cust_id = c.customer_id)
    AS orders
FROM customers.customers c
ORDER BY orders DESC NULLS LAST LIMIT 15;
```

Challenge 3: Two-Step CTE Pattern

```
-- High-spend customers in 2024
WITH high_spenders AS (
  SELECT cust_id, SUM(net_total) AS total
  FROM sales.orders
  WHERE EXTRACT(YEAR FROM order_date) = 2024
  GROUP BY cust_id
  HAVING SUM(net_total) > 10000
)
SELECT c.first_name, hs.total
FROM high_spenders hs
JOIN customers.customers c
  ON hs.cust_id = c.customer_id
ORDER BY hs.total DESC LIMIT 10;
```

Challenge 4: Recursive — Date Series

```
-- Last 7 days as a recursive series
WITH RECURSIVE last_7 AS (
  SELECT CURRENT_DATE - 6 AS day
  UNION ALL
  SELECT day + 1 FROM last_7
  WHERE day < CURRENT_DATE
)
SELECT day, TO_CHAR(day, 'Day')
  AS day_name
FROM last_7 ORDER BY day;
```

Quiz 1: What Makes a Subquery Correlated?

EASY

How can you tell at a glance that a subquery is correlated vs independent?

ANSWER: The inner query references a column from the OUTER table — usually via an alias like 'outer_e.dept_id' inside the inner WHERE. Without that reference, the inner query is independent and runs once. With it, the inner runs per outer row (logically).

Quiz 2: EXISTS vs IN — When?

MEDIUM

When should you prefer EXISTS over IN for the SAME logical question?

ANSWER: (1) When the inner query is expensive — EXISTS stops at first match. (2) For NOT EXISTS over NOT IN — handles NULLs safely. (3) When the inner query has potential duplicates — EXISTS doesn't care, IN materializes the whole list. EXPLAIN ANALYZE confirms which strategy wins on your data.

Quiz 3: CTE vs Derived Table

MEDIUM

Is there any DIFFERENCE between a CTE and a derived table from PostgreSQL's perspective?

ANSWER: Since PostgreSQL 12, NO meaningful difference — CTEs are inlined just like derived tables. Same execution plan, same performance. Choose CTE for READABILITY (multi-step logic, reuse within a query). Choose derived table for inline one-shot pre-aggregation.

Quiz 4: Recursive CTE Termination

HARD

ANSWER: This is INFINITE recursion! There's no WHERE clause to stop the recursive step. PostgreSQL will eventually error out (with stack depth or memory exhaustion), but the query may run for a while first. Always include a stop condition: WHERE n < 100.

Question

✅ ANSWER

```
WITH RECURSIVE x AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM x
)
SELECT * FROM x;
```

Quiz 5: NOT IN vs NOT EXISTS Behavior

CRAZY

Inner query returns: [1, 2, NULL, 4]. Outer value = 3. What does NOT IN return? NOT EXISTS?

ANSWER: NOT IN returns UNKNOWN (treated as FALSE) — because of the NULL in the inner list, NOT IN can't conclusively say 3 isn't in the list. NOT EXISTS returns TRUE — it just checks if any row matches; the NULL in the inner result doesn't affect the existence check. This is why NOT EXISTS is the safer pattern.

COMMON MISTAKE

EXISTS Without Correlation

The Mistake:
WHERE EXISTS (SELECT 1 FROM orders) — no reference to outer customer. EXISTS returns TRUE for every row (because orders has data). The filter does nothing.

The Fix:
EXISTS requires correlation in the inner WHERE: WHERE o.cust_id = c.customer_id. Without it, EXISTS just checks 'does the table have rows', not 'does THIS customer have orders'.

COMMON MISTAKE

Correlated Subquery Performance

The Mistake:
Putting the same expensive correlated subquery in both SELECT and WHERE — the database may run it twice per row. On 100K customers, that's 200K executions of an already-slow subquery.

The Fix:
Promote the correlated subquery to a CTE that runs once, then reference its result twice. Or use window functions (Database Concepts) for per-group aggregations. Always EXPLAIN ANALYZE before assuming correlation is the right tool.

COMMON MISTAKE

CTE Referenced Out of Order

The Mistake:
Defining CTE B that references CTE A, but writing them in wrong order: WITH B AS (... FROM A), A AS (...). PostgreSQL errors: 'relation A does not exist'.

The Fix:
CTEs must be defined in order — a CTE can reference earlier CTEs but not later ones. Write your CTE chain from the BASE up: foundation CTE first, then ones that build on it. The main SELECT is always LAST.

COMMON MISTAKE

Recursive CTE Without Stop Condition

The Mistake:
Recursive step that never returns zero rows — infinite recursion. PostgreSQL eventually errors out, but the query can hang for a while first.

The Fix:
ALWAYS include a WHERE clause that eventually becomes FALSE: WHERE n < 100, WHERE day < CURRENT_DATE. Add a safety counter (depth < 50) for hierarchies in case of unexpected cycles in the data.

INTERVIEW

Correlated vs Independent

Q: 'What's a correlated subquery?'

A: A subquery whose inner WHERE clause references a column from the OUTER query. It logically runs ONCE PER OUTER ROW. Independent subqueries run once and substitute their result. Modern optimizers often rewrite correlated subqueries as JOINs for performance.

INTERVIEW

EXISTS vs IN

Q: 'When EXISTS, when IN?'

A: (1) Use EXISTS for existence checks on large inner tables — it stops at first match. (2) Use NOT EXISTS instead of NOT IN to avoid the NULL trap. (3) Use IN when the inner query is small or a static list. The optimizer often picks the same plan either way; choose based on clarity and NULL safety.

INTERVIEW

Why CTEs Over Subqueries?

Q: 'When would you prefer a CTE to a nested subquery?'

A: When the query has 3+ logical steps that build on each other. CTEs let you name each step, read top-to-bottom, and reference earlier results. Nested subqueries read inside-out and are unreadable past 2 levels. PostgreSQL 12+ inlines CTEs, so there's no performance penalty.

INTERVIEW

Anatomy of a Recursive CTE

Q: 'Walk me through the structure of a recursive CTE.'

A: Three parts. (1) ANCHOR — a SELECT that returns the seed rows (the 'starting point'). (2) UNION ALL connecting anchor to recursive part. (3) RECURSIVE — a SELECT that references the CTE itself, producing the next generation. PostgreSQL repeats the recursive step until it returns zero rows.

INTERVIEW

Real Interview: Find Customers Above Department Average

Q: 'How many ways can you write: employees earning above their department average?'

A: At least three. (1) Correlated subquery in WHERE — inner uses outer.dept_id. (2) Derived table + JOIN — pre-aggregate per dept, then JOIN and compare. (3) Window function (Database Concepts) — AVG OVER (PARTITION BY dept_id) in SELECT, then filter. All three produce the same result; performance varies.

INTERVIEW

When to Use Recursive CTE?

Q: 'Give a real-world use case for recursive CTEs.'

A: (1) Org chart traversal — walk up/down employee-manager hierarchy. (2) Category tree — root category → subcategories → leaf categories. (3) Bill of materials — assembled product → components → raw materials. (4) Date series for gap-filling. (5) Path-finding in graph data. Any 'walk through connected data' problem.

INTERVIEW

Spot the Bug — Multiple Rows in Scalar

Q: 'A junior wrote WHERE salary > (SELECT salary FROM employees WHERE dept=5). It fails. What's wrong?'

A: The subquery returns multiple rows (dept 5 has multiple employees), but the > operator expects a scalar. PostgreSQL throws 'more than one row returned'. Fix: aggregate (SELECT MAX(salary)) or change to IN. The choice depends on intent — > MAX = strictly greater than every dept 5 salary; IN = matches any dept 5 salary.

KEY TAKEAWAYS

KEY TAKEAWAYS

Correlated Subqueries:

1. Inner query references outer row — runs per-row (logically).
2. Used for per-row dynamic benchmarks.
3. Can be slow; modern optimizers often rewrite as JOINs.

EXISTS / NOT EXISTS:

4. EXISTS stops at first match — faster than IN for large data.
5. NOT EXISTS is the safer anti-join — handles NULLs correctly.
6. SELECT 1 inside EXISTS is convention; value is ignored.

CTEs (WITH clause):

7. Named subqueries — readable top-to-bottom like a recipe.
8. Multiple CTEs chain; later ones reference earlier ones.
9. Modern replacement for nested subqueries. dbt and all BI tools use them.

Recursive CTEs:

10. Anchor + UNION ALL + recursive step. ALWAYS add a stop condition.

REAL WORLD

What You Achieved Today

You learned the four senior-analyst patterns: correlated subqueries (per-row dynamic comparison), EXISTS (efficient existence checks), CTEs (composable multi-step queries), and recursive CTEs (hierarchies and series).

This concludes the 'building blocks' phase. With earlier sessions, you have every fundamental SQL tool. From tomorrow, we explore database CONCEPTS that analysts need to know (normalization, ACID, transactions), then review, then window functions for advanced analytics.

Tomorrow: DB Concepts for Analysts — normalization, ACID, transactions. The theory layer that grounds everything you've learned.

Subqueries Part 2 + CTEs

-- Correlated Subquery

```
WHERE col > (SELECT AVG(x)
  FROM t inner_t
  WHERE inner_t.k = outer_t.k)
-- Inner references OUTER column
```

-- EXISTS

```
WHERE EXISTS (
  SELECT 1 FROM t
  WHERE t.id = outer.id
)  -- fast existence check
```

-- NOT EXISTS (NULL-safe)

```
WHERE NOT EXISTS (
  SELECT 1 FROM t
  WHERE t.id = outer.id
)  -- safer than NOT IN
```

Subqueries Part 2 + CTEs

```
-- Single CTE
WITH cte AS (SELECT ... FROM ...)
SELECT * FROM cte
JOIN other ON ...

-- Multiple CTEs
WITH a AS (...),
  b AS (SELECT ... FROM a ...),
  c AS (SELECT ... FROM b ...)
SELECT * FROM c;

-- Recursive CTE
WITH RECURSIVE cte AS (
  anchor_query
  UNION ALL
  recursive_query  -- references cte
) SELECT * FROM cte;
```

CHALLENGE

CTEs Take-Home Challenge (1/3)

Build these queries on RetailMart V3:

Easy (EXISTS):

1.: Customers with at least one return (EXISTS through returns → orders).

2.: Products never bought (NOT EXISTS — NULL-safe anti-join).

CHALLENGE

CTEs Take-Home Challenge (2/3)

Medium (CTEs):

3.: Churn analysis CTE: customers with no order in 6+ months. Show name, last order date, days since.

4.: Two chained CTEs: top 10 stores by revenue, with their employee count joined in.

Hard (Refactoring):

5.: Refactor a 3-level nested subquery into chained CTEs. Same result, more readable.

6.: Find employees above their department's average using TWO approaches: correlated subquery and derived table. Compare row counts.

Crazy (Recursive):

7.: Recursive CTE: walk a synthetic 4-level org chart (use VALUES) and show each employee with their level and path from CEO.

CHALLENGE

CTEs Take-Home Challenge (3/3)

Push your SQL file to GitHub!

https://github.com/starlordali4444/postgresql

STORY

Coming Tomorrow

Normalization (1NF, 2NF, 3NF) — why source schemas look the way they do. Denormalization tradeoffs — when stored aggregates are intentional. ACID (Atomicity, Consistency, Isolation, Durability) — what each means in plain English. Transactions — BEGIN, COMMIT, ROLLBACK at the concept level.

Database Concepts is the theory grounding for everything we've built. Concept-only — no procedural code, no hands-on locking. Just the principles every analyst needs to understand the data they query.